PIAZZA CLOUD SOFTWARE AS A SERVICE

Technical Report

Student name: [REPLACE WITH YOUR NAME]

Student ID: [REPLACE WITH YOUR STUDENT ID]

Module: Cloud Computing

Submission date: [REPLACE WITH DATE]

Repository: [REPLACE WITH YOUR GITHUB URL]

Declaration: This report describes the submitted implementation. Tutorial material was used as guidance and is acknowledged in the references. All screenshots, cloud addresses and deployment evidence inserted in the final version must come from the student's own execution.

Executive summary

Piazza is a RESTful Software as a Service that provides a small topic-based social discussion platform. Registered users authenticate using JSON Web Tokens, publish messages under Politics, Health, Sport or Tech, browse posts, and interact through likes, dislikes and comments. Every post has an expiry time. Expired posts remain available for historical browsing but no longer accept interactions. The service also identifies the live post with the greatest combined number of likes and dislikes for a selected topic.

The implementation uses Node.js, Express, MongoDB and Mongoose. It is packaged with Docker, deployable to a virtual machine, and includes Kubernetes resources for five replicas behind a load-balancing service. A test runner implements the twenty coursework scenarios involving Olga, Nick, Mary and Nestor.

1. Requirements and scope

The system implements the six principal Piazza actions: authenticated access, topic posting, topic browsing, liking/disliking/commenting, most-active-post lookup and expired-post history. All post resources are protected by bearer JWT authentication. Input is validated before database operations, and application rules prevent invalid or late interactions.

Requirement	Implementation	Evidence to insert
Authentication	Registration, login, bcrypt password hashing and JWT middleware	Login response and unauthorised request screenshot
Topic posts	Post model with title, topics, message, owner and expiry	Postman create-post response
Interactions	Like, dislike and embedded comment records	Updated post response
Expiry	Calculated Live/Expired status and interaction rejection	Expired-post error response
Most active	Likes plus dislikes used as the interest score	Most-active endpoint response
History	Expired posts filtered by topic and expiry date	Expired-history response

2. Architecture and design

The application follows a layered structure. Routes define HTTP resources, middleware authenticates and validates requests, controllers implement the business rules, Mongoose models define persistent data, and the configuration layer establishes the MongoDB connection. This separation keeps endpoint definitions short and makes the main rules easier to test and maintain.

Architecture flow:

```
Client / Postman
        | HTTPS/HTTP + Bearer JWT
Express routes
        |
Authentication and validation middleware
        |
Controllers and business rules
```

```
        |
Mongoose models
        |
MongoDB database
```

In the cloud deployment, the client reaches a Kubernetes LoadBalancer. The service distributes requests to five identical stateless API pods. All pods connect to the same externally reachable MongoDB database. JWT-based authentication allows any replica to verify a token without storing a server-side session.

3. Installation and project structure

The software requires Node.js 18 or later, npm and access to MongoDB. During local development, the application may connect to a local MongoDB service or MongoDB Atlas. Docker Compose provides another reproducible local option by starting both the API and a MongoDB container.

The main folders are:

src/config - database connection.

src/controllers - authentication and post behaviour.

src/middleware - JWT verification, validation and error handling.

src/models - User and Post schemas.

src/routes - REST resource definitions.

tests - automated twenty-case scenario.

postman - manual API collection.

kubernetes - five-replica deployment and LoadBalancer.

evidence - student-generated screenshots.

[INSERT YOUR OWN SCREENSHOT: project folder structure]

4. Authentication and validation

Registration accepts a name, email address and password. The password is hashed with bcrypt before storage. Login compares the submitted password with the stored hash and returns a signed JWT containing the user identifier and name. Protected routes require an Authorization header in the form “Bearer token”. The middleware verifies the signature and loads the corresponding user before the request reaches a controller.

Validation is performed with express-validator. Names, email addresses, password lengths, post titles, messages, topics, expiry timestamps and comment text are checked. Invalid requests return HTTP 400 with field-level information. Duplicate registration returns 409, incorrect credentials return 401, forbidden owner reactions return 403, missing posts return 404, and expired or duplicate interactions return 409.

5. Database design

5.1 User model

- name: required display name
- email: required, unique and normalised
- passwordHash: bcrypt hash excluded from normal query results
- createdAt and updatedAt: automatic timestamps

5.2 Post model

- title and message
- topics: one or more values from Politics, Health, Sport and Tech
- owner and ownerName
- expiresAt and automatic createdAt/updatedAt
- likes and dislikes: embedded records containing user, userName and createdAt
- comments: embedded records containing user, userName, text and createdAt
- virtual values: status, likeCount, dislikeCount and interestScore

Embedding interactions is suitable for the coursework-scale system because each post and its visible interactions can be returned in one query. User identifiers are retained to enforce ownership and duplicate-reaction rules. For a very high-volume production platform, interactions could be moved to separate collections to avoid unbounded document growth.

6. REST API resources

Method	Resource	Authentication	Purpose
POST	/api/auth/register	No	Create a user
POST	/api/auth/login	No	Return a JWT
GET	/api/auth/profile	Yes	Return current user
POST	/api/posts	Yes	Create a post
GET	/api/posts	Yes	Browse all posts
GET	/api/posts/:postId	Yes	Browse one post
GET	/api/posts/topic/:topic	Yes	Browse by topic
POST	/api/posts/:postId/like	Yes	Like a live post
POST	/api/posts/:postId/dislike	Yes	Dislike a live post
POST	/api/posts/:postId/ comments	Yes	Comment on a live post
GET	/api/posts/topic/:topic/ expired	Yes	Browse expired history
GET	/api/posts/topic/:topic/ most-active	Yes	Find highest-interest live post

6.1 Example request

```
POST /api/posts
Authorization: Bearer <JWT>
{
  "title": "Responsible artificial intelligence",
  "topics": ["Tech"],
  "message": "A discussion about responsible AI services.",
  "expiresAt": "2030-01-01T12:00:00.000Z"
}
```

7. Application rules

- A post must expire in the future when it is created.
- Expired posts remain browsable but reject likes, dislikes and comments.
- A post owner cannot like or dislike their own post.
- A user cannot apply the same reaction twice.
- Changing from like to dislike, or vice versa, removes the previous reaction.
- The most-active endpoint considers only live posts and ranks them by likes plus dislikes.
- All post browsing and interaction endpoints require authentication.

8. Testing

The `tests/courseworkTestRunner.js` script performs the twenty required scenarios against a running API. It uses unique email addresses on each execution, stores each user's token, records created post identifiers and checks status codes and response data. The Health post expires after 3.5 seconds to make the expiry test practical.

TC	Scenario	Expected result
1-2	Four users register and obtain tokens	Success for every user
3	Olga browses without token	401 unauthorised
4-6	Olga, Nick and Mary create Tech posts	Three live posts
7	Browse initial Tech posts	Zero interactions
8-10	Apply required likes/dislike and browse	Mary 2/1; Nick 1/0
11	Mary likes own post	403 forbidden
12-13	Round-robin comments and browse	Four comments
14-16	Create/browse/comment on Health post	One comment before expiry
17	Dislike after expiry	409 conflict
18	Browse expired Health post	Expired with one comment
19	Expired Sports history	Empty result
20	Most-active Tech post	Mary's post

[INSERT YOUR OWN SCREENSHOT: console showing "Result: 20 passed, 0 failed"]

[INSERT REPRESENTATIVE POSTMAN SCREENSHOTS OR A CLEAR TEST EVIDENCE TABLE]

9. Docker and virtual machine deployment

The Dockerfile uses a lightweight Node.js image, installs production dependencies, runs as a non-root user and exposes port 3000. Docker Compose starts the API and MongoDB for local demonstration. For cloud deployment, the image can be built and pushed to Docker Hub, or the repository can be cloned into a Google Cloud virtual machine and built there.

Final evidence must show the student's own VM, repository deployment, running container and API response through the VM external IP address.

[INSERT YOUR OWN SCREENSHOT: VM instance and external IP]

[INSERT YOUR OWN SCREENSHOT: Docker container running]

[INSERT YOUR OWN SCREENSHOT: /health response through VM IP]

10. Kubernetes deployment

The Kubernetes Deployment defines five API replicas. Environment secrets are supplied through a Kubernetes Secret rather than embedded in source code. A Service of type LoadBalancer exposes the replicas. Readiness and liveness probes call the `/health` endpoint so Kubernetes can avoid routing traffic to an unhealthy container and restart it if necessary.

[INSERT YOUR OWN SCREENSHOT: kubectl get pods showing five Running replicas]

[INSERT YOUR OWN SCREENSHOT: kubectl get service showing external LoadBalancer address]

[INSERT YOUR OWN SCREENSHOT: public API response from the load balancer]

11. Quality, security and complexity

Code is separated by responsibility and uses descriptive function names, consistent indentation, comments around non-obvious behaviour and central error handling. Secrets are excluded from Git. Password hashes are never returned in ordinary queries. Helmet adds common HTTP security headers, and request validation reduces malformed input.

Most database lookups use indexed MongoDB identifiers or simple topic/expiry filters. Returning a topic's posts is O(n) in the number of matching posts and serialising their embedded interactions. Selecting the most-active post currently loads the live topic posts and sorts them, which is O(n log n). A production optimisation would use an aggregation pipeline with computed scores and suitable compound indexes.

12. Limitations and future work

- Refresh tokens and explicit token revocation
- Rate limiting and account lockout
- Pagination for posts and comments
- Separate interaction collections for high-volume scaling
- Automated Jest/Supertest integration tests in CI
- Frontend client and accessibility testing
- Observability through structured logs, metrics and tracing
- Infrastructure as code and CI/CD deployment

13. Conclusion

The Piazza service demonstrates cloud-oriented REST development, secure user access, persistent NoSQL data, rule-based interactions, automated scenario testing, container packaging and replicated Kubernetes deployment. The design meets the principal functional requirements while remaining readable and extendable. The final submitted report must be updated with genuine screenshots and identifiers from the student's own GitHub, VM, Docker and Kubernetes environments.

References

Express.js (2026) Express web framework documentation. Available at: https://expressjs.com/ (Accessed: [date]).

Jones, M., Bradley, J. and Sakimura, N. (2015) JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force.

MongoDB, Inc. (2026) MongoDB documentation. Available at: https://www.mongodb.com/docs/ (Accessed: [date]).

Kubernetes Authors (2026) Kubernetes documentation. Available at: https://kubernetes.io/docs/ (Accessed: [date]).

Docker, Inc. (2026) Docker documentation. Available at: https://docs.docker.com/ (Accessed: [date]).

Warestack (2023-2024) Cloud Computing tutorial repository. GitHub repository supplied for module laboratory guidance. Available at: https://github.com/warestack/cc (Accessed: [date]).

Appendix A: Final evidence checklist

☐ Student name/ID/date completed

☐ GitHub repository URL inserted

☐ 20-case output captured

☐ Postman evidence inserted

☐ VM external IP evidence inserted

☐ Docker build/run evidence inserted

☐ Five Kubernetes replicas shown

☐ LoadBalancer external address shown

☐ All placeholders removed

☐ References access dates completed

☐ No secrets committed